

Reto Hackathon — Almacén inteligente para taller

Oct 6, 2026 · @Fernando Vazquez

1. Contexto y objetivo

Construyan el servicio de inventario de un taller de reparación de equipo industrial (husillos, motores eléctricos). Debe enterarse de lo que pasa en el taller por eventos y responder con existencias, reservas y faltantes en tiempo real.

Hoy el taller ya registra cada equipo que entra (una **orden de trabajo**), lo inspecciona y genera una lista de partes: cuáles están bien, cuáles están dañadas y cuáles faltan, y si hay que comprarlas o repararlas. Compras recibe material. Pero nadie sabe qué hay en el almacén, así que cada pieza se busca a mano y se pide tarde.

El servicio que construyan debe cerrar este ciclo:

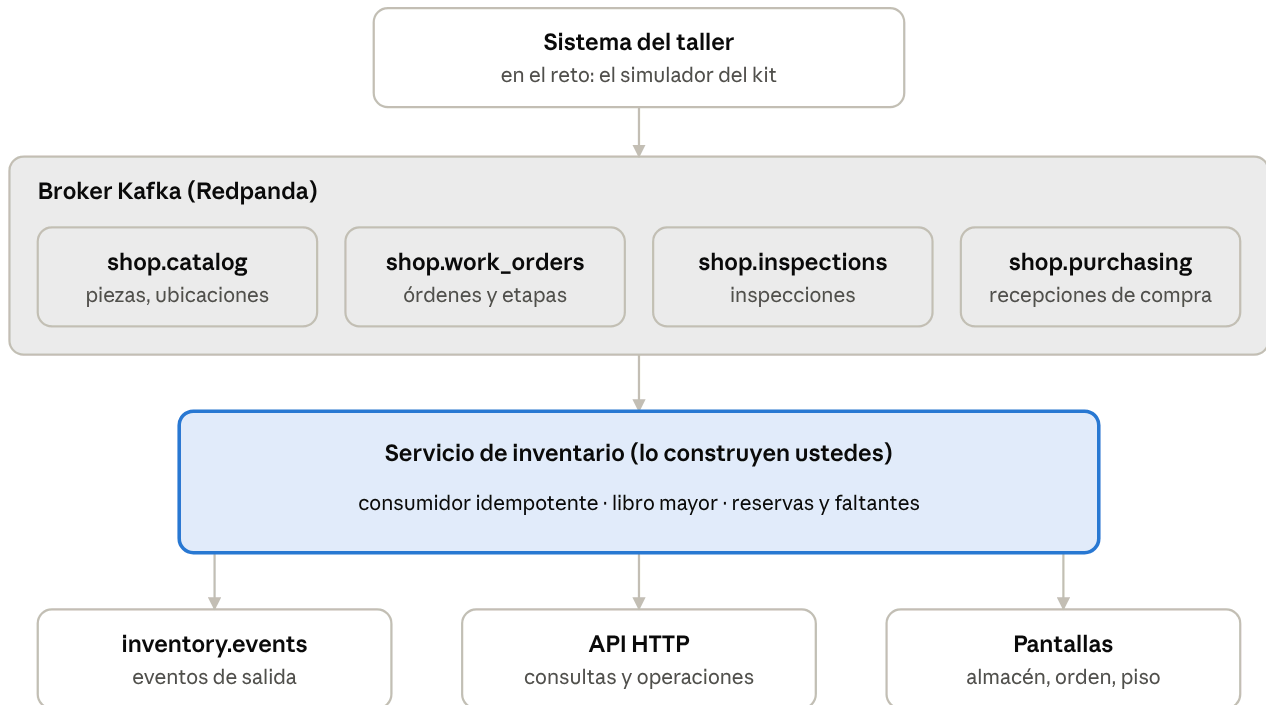
1. Una inspección aprobada pide piezas para una orden de trabajo.
2. Si hay existencia, se **reserva** para esa orden.
3. Si no alcanza, se registra un **faltante** y se sugiere una compra.
4. Cuando llega la compra, entra al almacén y se surten las reservas pendientes.
5. El técnico retira la pieza contra su orden y el stock baja.

No tienen que construir el sistema del taller: ese ya existe y les manda los eventos. Ustedes construyen el inventario que los consume.

2. Arquitectura y transporte

El taller publica eventos en un broker Kafka y su servicio los consume. Ustedes no leen la base de datos del taller ni llaman a su API: todo lo que necesitan llega por los tópicos.

El inventario solo escucha eventos del taller y publica los suyos



arquitectura · 4 tópicos de entrada, 3 salidas

Garantías de entrega, tal como funcionará con el sistema real:

- **Al menos una vez.** Un evento puede llegar repetido; deduplicar por `event_id` es responsabilidad suya.
- **Orden por llave.** Dentro de un tópico, los eventos con la misma `key` llegan en orden. Entre tópicos distintos no hay orden garantizado.
- **Retención completa.** Los tópicos guardan todo el historial, así que pueden releer desde el inicio para reconstruir su estado.
- **Errores.** Un evento que no puedan procesar después de reintentar va a un tópico `inventory.dlq` propio, sin detener el consumo.

3. Datos de entrada

Todos los eventos llegan en JSON con el mismo sobre; lo que cambia es `data`. Las fechas van en ISO 8601 UTC, los IDs son enteros y los importes son decimales en texto.

```
{
  "event_id": "0b9f3c1e-7a42-4c1d-9e0a-5d2f8b61c7aa",
  "event_type": "inspection.approved",
  "event_version": 1,
  "occurred_at": "2026-10-06T16:42:11Z",
  "source": "shop-floor",
  "key": "R061026012",
  "data": { }
}
```

`event_id` es único y sirve para deduplicar. `key` es la llave de partición: el código de la orden de trabajo, o el ID con el prefijo de su entidad en los eventos de catálogo (part:305, location:14, model:4).

Catálogo de eventos

Evento	Tópico	Cuándo llega	Campos de data
<code>part.upserted</code>	<code>shop.catalog</code>	Alta o edición de una pieza; todas al arrancar	<code>part_id</code> , <code>sku</code> (puede ser null), <code>name</code> , <code>family</code> , <code>group</code> , <code>subgroup</code> , <code>unit {id, name}</code> (null), <code>image_url</code> (null), <code>active</code>
<code>location.upserted</code>	<code>shop.catalog</code>	Alta o edición de una ubicación	<code>location_id</code> , <code>code</code> (código de barras, ej. U-014), <code>name</code> , <code>is_workbench</code> , <code>active</code>

Evento	Tópico	Cuándo llega	Campos de data
<code>bom.upserted</code>	<code>shop.catalog</code>	Cambio en la lista de partes de un modelo	<code>model_id</code> , <code>model_name</code> , <code>brand</code> , <code>lines[]</code> : <code>bom_line_id</code> , <code>part_id</code> (null), <code>name</code> , <code>group_name</code> (null), <code>unit</code> (null), <code>qty_per_unit</code> (null), <code>sort_order</code>
<code>work_order.opened</code>	<code>shop.work_orders</code>	Entra un equipo al taller	<code>work_order_id</code> , <code>code</code> , <code>model_id</code> , <code>department</code> , <code>received_at</code>
<code>work_order.stage_changed</code>	<code>shop.work_orders</code>	Avanza o retrocede de etapa	<code>work_order_id</code> , <code>from_stage</code> , <code>to_stage</code>
<code>work_order.deleted</code>	<code>shop.work_orders</code>	Se da de baja un registro erróneo	<code>work_order_id</code> , <code>code</code>
<code>inspection.submitted</code>	<code>shop.inspections</code>	El técnico termina su inspección	<code>inspection_id</code> , <code>work_order_id</code> , <code>kind</code> (<code>quick</code> / <code>full</code>), <code>items[]</code>
<code>inspection.approved</code>	<code>shop.inspections</code>	El supervisor la aprueba (este dispara las reservas)	igual que <code>submitted</code> + <code>approved_by</code> , <code>approved_at</code>
<code>inspection.rejected</code>	<code>shop.inspections</code>	El supervisor la devuelve	<code>inspection_id</code> , <code>work_order_id</code> , <code>reason</code>

Evento	Tópico	Cuándo llega	Campos de data
<code>inspection.discarded</code>	<code>shop.inspections</code>	Se cancela sin aprobar	<code>inspection_id</code> , <code>work_order_id</code>
<code>inspection.voided</code>	<code>shop.inspections</code>	Se anula una inspección que ya estaba aprobada (la orden regresa de etapa)	<code>inspection_id</code> , <code>work_order_id</code> , <code>reason</code>
<code>purchase.item_received</code>	<code>shop.purchasing</code>	Compras recibe una línea de una requisición	<code>purchase_request_id</code> , <code>request_number</code> , <code>line_id</code> , <code>part_number</code> (texto libre), <code>description</code> , <code>quantity</code> , <code>unit_price</code> , <code>currency</code> , <code>purpose</code> (restock / customer_order), <code>received_at</code> , <code>work_order_code</code> (null)

Cada elemento de `items[]` trae `inspection_item_id`, `bom_line_id`, `part_id` (null), `name`, `group_name` (null), `condition` (ok / damaged / missing), `action` (buy / repair / null), `quantity` (null) y `note` (null). Los valores de `department` son `spindles`, `motors`, `electronics` y `torque_center`. Las etapas, en orden, son: `awaiting_quick_inspection`, `quick_inspection`, `awaiting_full_inspection`, `full_inspection`, `cleaning`, `assembly`, `run_in`, `final_assembly` y `final_release`.

Ejemplos

```
{ "event_type": "inspection.approved", "key": "R061026012", "data": {
  "inspection_id": 418, "work_order_id": 1203, "kind": "quick",
  "approved_by": 12, "approved_at": "2026-10-06T16:42:11Z",
  "items": [
    { "inspection_item_id": 9001, "bom_line_id": 77, "part_id": 305, "name":
"RODAMIENTO DELANTERO", "group_name": null,
      "condition": "damaged", "action": "buy", "quantity": 2, "note": "pista
con picaduras" },
    { "inspection_item_id": 9002, "bom_line_id": 81, "part_id": null, "name":
"KIT DE ORINGS", "group_name": null,
      "condition": "missing", "action": "buy", "quantity": 13, "note": null
    },
    { "inspection_item_id": 9003, "bom_line_id": 85, "part_id": 412, "name":
"FLECHA", "group_name": null,
      "condition": "ok", "action": null, "quantity": null, "note": null }
  ] } }
```

```
{ "event_type": "purchase.item_received", "key": "PR-2044", "data": {
  "purchase_request_id": 88, "request_number": "2044", "line_id": 540,
  "part_number": "7014 CTA/P4", "description": "Rodamiento de contacto
angular",
  "quantity": 4, "unit_price": "1850.00", "currency": "MXN", "purpose":
"restock",
  "received_at": "2026-10-08T15:10:00Z", "work_order_code": "R061026012" } }
```

Lo que deben saber de los datos

Así llegan los datos reales; no hay una versión "limpia".

- **Pieza sin identificar.** Una línea puede venir con `part_id: null` (solo nombre libre). No se puede reservar automáticamente: repórtenla como faltante sin identificar.
- **Compras sin ID de pieza.** `purchase.item_received` trae un `part_number` en texto, no un `part_id`. Tienen que relacionarlo con el `sku` del catálogo (puede variar en espacios o mayúsculas) y mandar a revisión manual lo que no coincida.
- **Kits.** Las líneas de un modelo con el mismo `group_name` forman un kit; si `group_name` es null, el kit lo forman las líneas con el mismo `name`. Decidan si se inventaría el kit, sus componentes o ambos.

- **Rechazos.** Después de un `inspection.rejected` llega una inspección nueva (otro `inspection_id`) sobre la misma orden, normalmente con las mismas piezas. Solo una aprobación genera reservas.
- **Varias inspecciones por orden.** Una orden puede tener varias aprobadas (una rápida y luego varias completas). No dupliquen reservas por la misma pieza y la misma orden.
- **Duplicados y desorden.** Un evento puede llegar dos veces, y entre tópicos distintos no hay orden garantizado (una inspección puede llegar antes que el `part.upserted` de su pieza).
- **Inspecciones anuladas.** Una inspección ya aprobada puede anularse después con `inspection.voided` (la orden regresó de etapa). Sus reservas activas se liberan y sus faltantes abiertos se cierran.
- **Cantidad sin capturar.** Una línea con `action = buy` puede traer `quantity: null`. Usen el `qty_per_unit` de su línea en la lista de partes (`bom_line_id`); si también es null, repórtenla como faltante con `missing_quantity: null`.
- **Inspecciones completas sin piezas.** Una inspección `full` puede llegar con `items: []`. Se procesa igual (aprobación, rechazo, anulación), solo que no genera reservas.
- **Compras para un cliente.** Una recepción con `purpose = customer_order` es material comprado para revender a un cliente: no entra como existencia del taller ni surte faltantes. Solo `restock` entra al almacén.

4. Qué deben construir

Un servicio que consuma los eventos, guarde el inventario como un **libro mayor de movimientos** y exponga existencias, reservas y faltantes. El stock nunca se edita directo: siempre se calcula como la suma de los movimientos.

Modelo mínimo

Concepto	Qué representa
Movimiento	Entrada, salida, transferencia o ajuste de una pieza en una ubicación. Es inmutable: un error se corrige con otro movimiento
Saldo	Por pieza y ubicación: <code>on_hand</code> , <code>reserved</code> y <code>available = on_hand - reserved</code> . Es una proyección de los movimientos

Concepto	Qué representa
Reserva	Cantidad apartada de una pieza para una orden de trabajo, con estado <code>active</code> , <code>fulfilled</code> o <code>cancelled</code>
Faltante	Lo que una orden necesita y no hay, o una pieza sin identificar
Política	Mínimo, máximo, cantidad de reorden y tiempo de entrega por pieza

Flujos

1. **Recepción.** Con `purchase.item_received` y `purpose = restock`, relacionen el `part_number` con el catálogo y registren una entrada en una ubicación de recepción. Si trae `work_order_code`, surtan primero el faltante de esa orden. Las de `purpose = customer_order` no entran al inventario.
2. **Reserva automática.** Con `inspection.approved`, por cada línea con `action = buy` reserven lo disponible. Lo que no alcance se vuelve faltante.
3. **Sugerencia de compra.** Agrupen los faltantes y las piezas bajo mínimo en una sugerencia de reabasto por pieza, con la cantidad y las órdenes que la piden.
4. **Salida contra orden.** El técnico escanea la ubicación y la pieza y retira contra un código de orden. La reserva pasa a `fulfilled`.
5. **Transferencia.** Mover piezas entre ubicaciones escaneando origen y destino.
6. **Conteo cíclico.** Contar una ubicación, comparar contra el sistema y registrar un ajuste con motivo obligatorio.
7. **Liberación.** Si una orden se da de baja (`work_order.deleted`) o una inspección aprobada se anula (`inspection.voided`), liberen las reservas activas que generó y cierren sus faltantes abiertos.

Reglas obligatorias

- **Idempotencia.** Procesar dos veces el mismo `event_id` no cambia nada. Lo vamos a probar reenviando eventos.
- **Sin stock negativo.** Una salida o reserva nunca deja `available` por debajo de cero, salvo un ajuste autorizado.

- **Concurrencia.** Dos aprobaciones simultáneas no pueden reservar la misma última pieza.
- **Tolerancia al desorden.** Si llega una inspección que referencia una pieza que aún no conocen, no se pierde: se reintenta o se encola.
- **Reconstruible.** Borrando la base y releendo los tópicos desde el inicio, deben llegar al mismo saldo.
- **Piezas del cliente fuera del inventario.** Una línea con `action = repair` es una pieza del propio equipo del cliente que se va a reparar: no es stock del taller, no se reserva y no entra como existencia.

5. Lo que debe salir

Se necesitan tres salidas: eventos de inventario en un tópico propio, una API HTTP y pantallas para almacén y piso. Los eventos y la API se evalúan con pruebas automáticas, así que respeten nombres y formas.

Eventos de salida (tópico `inventory.events`)

Usan el mismo sobre de la sección 3, con `source: "inventory"` .

Evento	Cuándo	Campos de data
<code>stock.received</code>	Entra material por compra	<code>part_id</code> , <code>location_id</code> , <code>quantity</code> , <code>unit_cost</code> , <code>currency</code> , <code>purchase_line_id</code>
<code>stock.unmatched_receipt</code>	Una recepción no coincide con el catálogo	<code>purchase_line_id</code> , <code>part_number</code> , <code>description</code> , <code>quantity</code>
<code>stock.reserved</code>	Se aparta stock para una orden	<code>reservation_id</code> , <code>work_order_id</code> , <code>part_id</code> , <code>quantity</code> , <code>inspection_item_id</code>
<code>stock.shortage_detected</code>	Una orden pide más de lo disponible	<code>work_order_id</code> , <code>part_id</code> (null), <code>name</code> , <code>missing_quantity</code> , <code>inspection_item_id</code>
<code>stock.shortage_resolved</code>	Llega material y cubre el faltante	<code>work_order_id</code> , <code>part_id</code> , <code>quantity</code>

Evento	Cuándo	Campos de data
<code>stock.issued</code>	Se retira pieza contra una orden	<code>work_order_id</code> , <code>part_id</code> , <code>location_id</code> , <code>quantity</code> , <code>issued_by</code>
<code>stock.transferred</code>	Movimiento entre ubicaciones	<code>part_id</code> , <code>from_location_id</code> , <code>to_location_id</code> , <code>quantity</code>
<code>stock.adjusted</code>	Ajuste por conteo	<code>part_id</code> , <code>location_id</code> , <code>delta</code> , <code>reason</code>
<code>stock.reorder_suggested</code>	Una pieza baja del mínimo o acumula faltantes	<code>part_id</code> , <code>suggested_quantity</code> , <code>work_order_ids[]</code>

```
{ "event_type": "stock.shortage_detected", "source": "inventory", "key":
"R061026012", "data": {
  "work_order_id": 1203, "part_id": 305, "name": "RODAMIENTO DELANTERO",
  "missing_quantity": 1, "inspection_item_id": 9001 } }
```

API mínima

Método y ruta	Responde
GET /parts/{part_id}/availability	Saldo por ubicación: on_hand , reserved , available
GET /parts/{part_id}/ledger	Kardex: movimientos con saldo corrido, del más reciente al más antiguo
GET /work-orders/{code}/materials	Por línea de inspección: reservado, entregado y faltante
GET /shortages	Faltantes abiertos, filtrables por orden o pieza
GET /reorder-suggestions	Sugerencias de compra vigentes
GET /unmatched-receipts y POST /unmatched-receipts/{id}/resolve	Recepciones por relacionar a mano con una pieza
POST /issues , POST /transfers , POST /counts	Salida, transferencia y conteo cíclico

Pantallas

- **Almacén.** Existencias por pieza y ubicación, faltantes abiertos y sugerencias de compra.
- **Orden de trabajo.** Busca por código y muestra qué piezas tiene reservadas, entregadas y pendientes.
- **Piso (móvil).** Escanear ubicación y pieza para retirar, transferir o contar. Debe funcionar con la cámara de un celular.
- **Kardex** de una pieza.

6. Procedimiento

Cada equipo recibe un kit de arranque que levanta el broker y el simulador del taller con un solo comando. Lenguaje, base de datos y framework de pantallas son libres.

Kit de arranque

- **Contrato (`contract/`).** Un JSON Schema por evento, la API en OpenAPI y las reglas exactas que revisan las pruebas (incluida la ubicación de recepción `U-100`). Si algo de este documento no coincide con el contrato, gana el contrato.
- **`docker-compose.yml`** con Redpanda (compatible con la API de Kafka) y Redpanda Console para ver los tópicos.
- **Simulador del taller.** Publica primero la carga inicial del catálogo y después reproduce seis semanas de operación, armadas sobre el catálogo y los diagnósticos reales del taller, a velocidad acelerada, con duplicados y desorden incluidos.
- **`events.jsonl`** . Los mismos eventos en un archivo, por si un equipo quiere desarrollar sin broker al principio.
- **Pruebas de contrato.** Un script que publica escenarios conocidos y revisa sus eventos de salida y su API. Pueden correrlo cuantas veces quieran.
- **Etiquetas impresas** de ubicaciones y piezas para la demo con escaneo real.

Pasos

1. Levanten el kit y confirmen que ven llegar los eventos en la consola.
2. Consuman el catálogo y construyan el libro mayor con recepciones y salidas.
3. Agreguen reservas y faltantes a partir de inspecciones aprobadas.
4. Corran las pruebas de contrato y corrijan hasta que pasen.
5. Construyan las pantallas y prueben el flujo de piso con un celular.
6. Preparen la demo.

Entregables

- Repositorio con código y un `README` que lo levante con un comando sobre el kit.
- Un documento de decisiones, de máximo una página, que responda: cómo relacionan compras con el catálogo, cómo manejan kits, cómo evitan reservas dobles y cómo garantizan idempotencia.
- Demo en vivo de 5 minutos. El jurado aprueba una inspección en el simulador; ustedes muestran la reserva, el faltante, la llegada de la compra y la salida escaneada.

Reglas

- No se modifica el simulador ni el formato de los eventos de entrada.

- Los eventos y rutas de la sección 5 respetan exactamente sus nombres y campos. Pueden agregar más, pero no cambiar esos.

7. Evaluación

Se califica sobre 100 puntos. Correctitud e integración se miden con las pruebas de contrato; lo demás lo evalúa el jurado en la demo.

Criterio	Puntos	Cómo se mide
Correctitud del libro mayor	25	Pruebas de contrato: saldos, idempotencia, sin negativos, reconstrucción desde cero
Integración con el taller	25	Reservas y faltantes correctos desde inspecciones, recepciones relacionadas, órdenes dadas de baja liberadas
Experiencia en piso	20	Escaneo con celular, pocos pasos para retirar o contar, claridad de la vista por orden
Decisiones justificadas	15	Documento de decisiones: kits, relación con compras, concurrencia
Extras	15	Ver niveles

Niveles

- **Base.** Libro mayor, recepción, salida con escaneo, kardex y saldos por ubicación.
- **Intermedio.** Reservas y faltantes automáticos, mínimos y máximos, conteo cíclico, recepciones sin relacionar.
- **Avanzado (extras).** Pronóstico de demanda por modelo a partir de las listas de partes y las inspecciones históricas, valuación del inventario a costo promedio, tablero de rotación (piezas críticas y piezas sin movimiento), observabilidad del consumidor (lag y errores).